SISTEMAS OPERACIONAIS

Deadlocks

Prof. Matheus A. Souza
Eng. Software/Sist. Informação
PUC Minas

Deadlocks

- Cenário atual:
  - Multiprogramação
  - *Multithreading*
  - Compartilhamento e concorrência por recursos

Deadlocks

- Cenário atual:
  - Multiprogramação
  - *Multithreading*
  - Compartilhamento e concorrência por recursos

- Recursos exclusivos: necessidade de sincronização e espera

Deadlocks

- Situação na qual nenhum processo em um grupo consegue avançar
  - *"Grupo de processos em bloqueio por causa de recursos que estão retidos por outros processos em bloqueio no mesmo grupo"*

Deadlocks

> Lei de trens do Kansas:

"When two trains approach each other at a crossing, both shall come to a full stop and neither shall start up again until the other has gone."

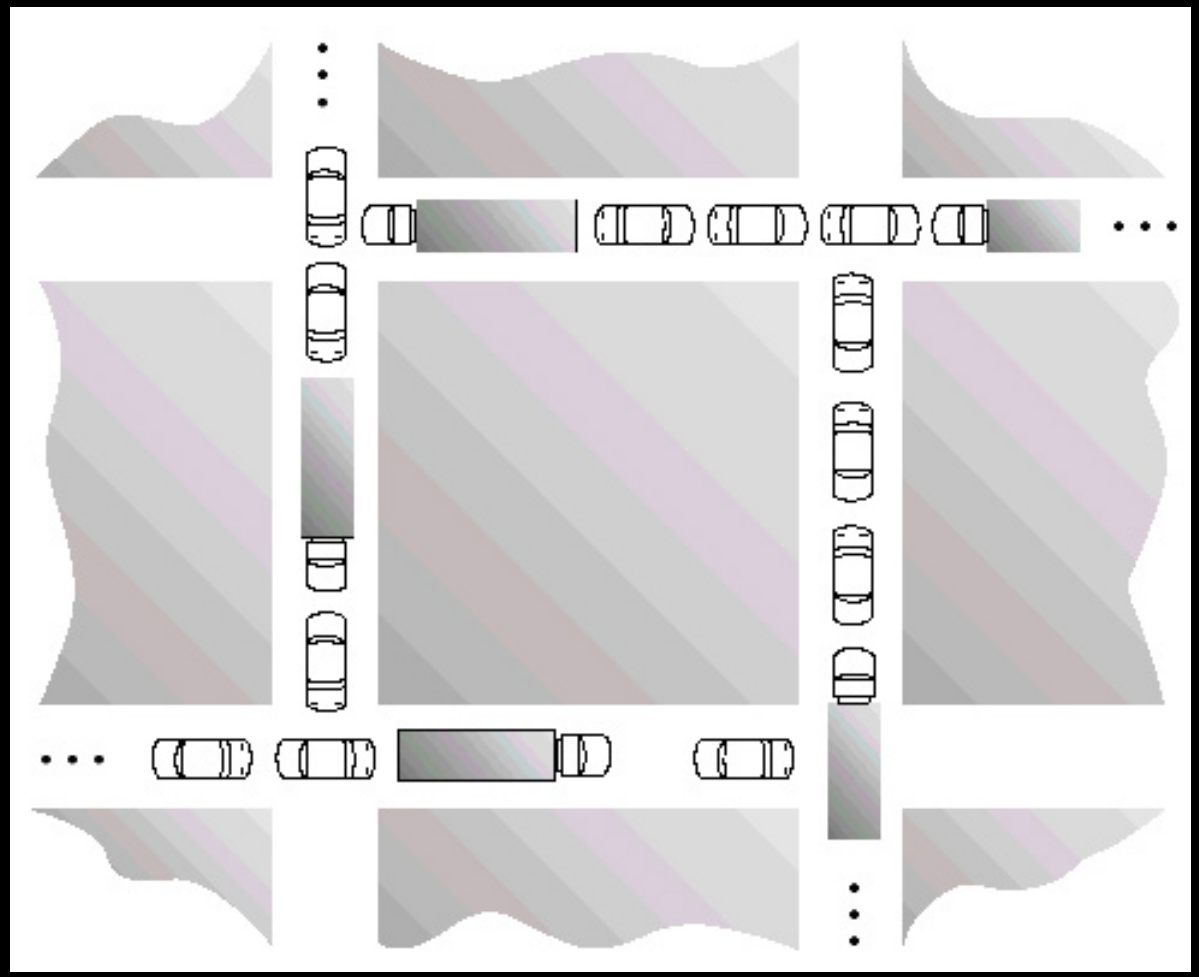
Deadlocks

> Lei de trens do Kansas:

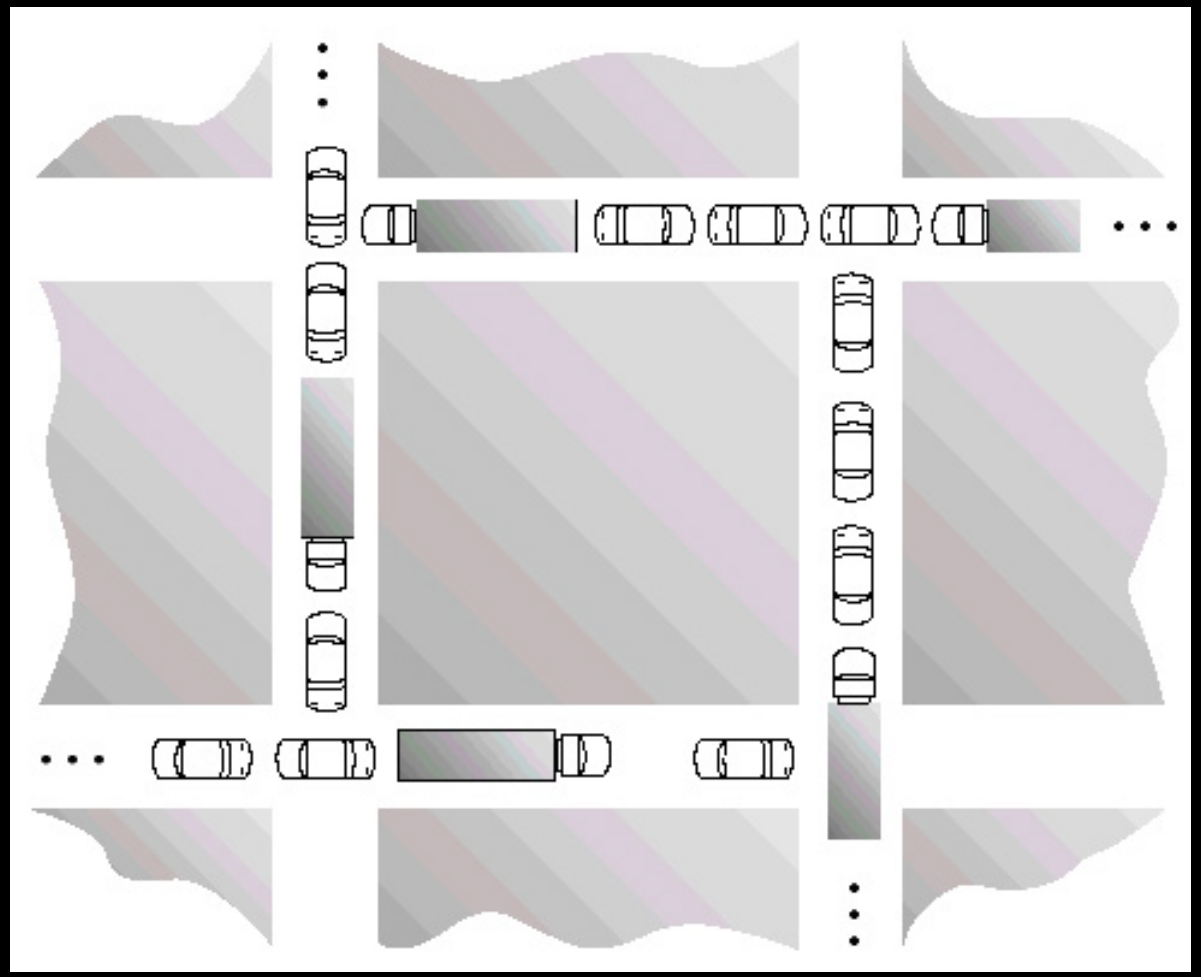
"When two trains approach each other at a crossing, both shall come to a full stop and neither shall start up again until the other has gone."

> A intenção era boa, mas... Implementação falha traz problemas.

Deadlocks



Deadlocks



São Paulo, 2 de fevereiro de 2017
https://www.facebook.com/radiobandnewsfm/photos/a.245524728892111.57638.245513265559924/1158652970912611/?type=3&theater

Ocorrência de *deadlocks*

- Necessitam-se quatro condições simultâneas:
  - Exclusão mútua
  - Retenção e espera (*hold and wait*)
  - Sem possibilidade de preempção
  - Espera circular

- Então, o SO...

Prevenção de deadlocks

Prevenção de *deadlocks*

> Prevenir: tomar atitudes para que algo não aconteça

> Prevenção de *deadlocks*: SO precisa garantir a eliminação de ao menos uma das quatro condições de ocorrência

Prevenção de *deadlocks*

- Prevenir exclusão mútua: impedir que haja acesso exclusivo a recursos compartilhados.

- É viável um sistema multiprogramado, *multithread*, sem recursos de acesso exclusivo?
  - Produtor/consumidor
  - Leitor/escritores
  - ...

Prevenção de *deadlocks*

- Prevenir retenção e espera: estabelecer protocolos para esperar (*wait*) e reter (*hold*) um recurso.

- Idéias:
  - Só é possivel manter/usar um recurso por vez (?)

Prevenção de *deadlocks*

- Prevenir retenção e espera: estabelecer protocolos para esperar (*wait*) e reter (*hold*) um recurso.

- Idéias:
  - ~~Só é possivel manter/usar um recurso por vez (?)~~
  - Alocar todos os recursos no início, ou não alocar nenhum

Prevenção de *deadlocks*

- Prevenir retenção e espera: estabelecer protocolos para esperar (*wait*) e reter (*hold*) um recurso.

- Idéias:
  - ~~Só é possivel manter/usar um recurso por vez (?)~~
  - Alocar todos os recursos no início, ou não alocar nenhum
    - Desperdício
    - *Starvation*

Prevenção de *deadlocks*

- Prevenir retenção e espera: estabelecer protocolos para esperar (*wait*) e reter (*hold*) um recurso.

- Idéias:
  - ~~Só é possivel manter/usar um recurso por vez (?)~~
  - ~~Alocar todos os recursos no início, ou não alocar nenhum~~
    - Desperdício
    - *Starvation*

Prevenção de *deadlocks*

- Permitir preempção de qualquer recurso: qualquer processo suspenso ou bloqueado seria obrigado a liberar seus recursos
- No retorno do bloqueio/suspensão, os recursos seriam recuperados

Prevenção de *deadlocks*

- Permitir preempção de qualquer recurso: qualquer processo suspenso ou bloqueado seria obrigado a liberar seus recursos
- No retorno do bloqueio/suspensão, os recursos seriam recuperados

- É possível fazer isso com segurança para qualquer recurso?

Prevenção de *deadlocks*

- Prevenir a espera circular: protocolos obrigando uma espera linear

- Ex: numerar os recursos e só permitir alocação ordenada
  - Sistema de arquivos: 2
  - Rede: 9
  - Impressora: 13

Prevenção de *deadlocks*

- Prevenir a espera circular: protocolos obrigando uma espera linear

- Ex: numerar os recursos e só permitir alocação ordenada
  - Sistema de arquivos: 2
  - Rede: 9
  - Impressora: 13

  - Fácil para o sistema...

Prevenção de *deadlocks*

- Prevenir a espera circular: protocolos obrigando uma espera linear

- Ex: numerar os recursos e só permitir alocação ordenada
  - Sistema de arquivos: 2
  - Rede: 9
  - Impressora: 13

  - Fácil para o sistema...
  - ... difícil para o programador.

Evitar os deadlocks

Evitar *deadlocks*

- Sabendo que há risco de ocorrência, analisar cada passo do sistema buscando evitar que aconteçam deadlocks

- Conceito de *estado seguro do sistema*:
  - SO pode alocar recursos a um processo se puder garantir que não acontecerá um deadlock

Evitar *deadlocks*

> Estado seguro tem idéia simples: evitar que os recursos sejam fornecidos em quantidade superior ao necessário para terminar todos os processos.

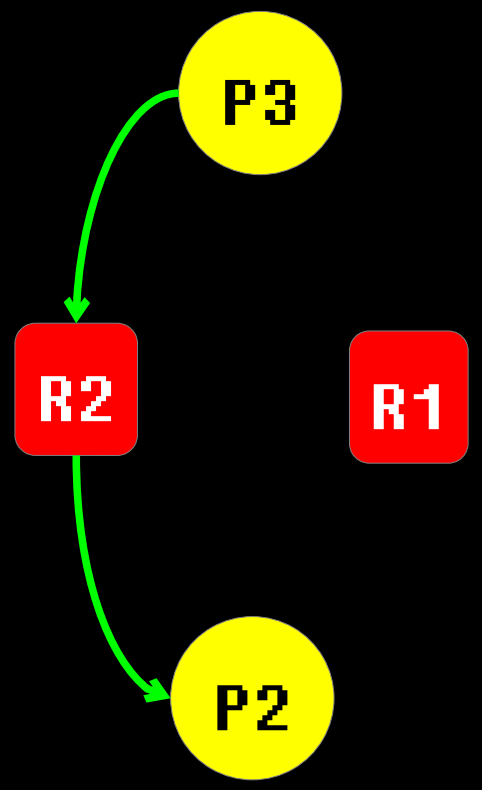
Evitar *deadlocks*

- Estado seguro tem idéia simples: evitar que os recursos sejam fornecidos em quantidade superior ao necessário para terminar todos os processos.

- Alternativa 1: usando grafos de alocação e evitando ciclos

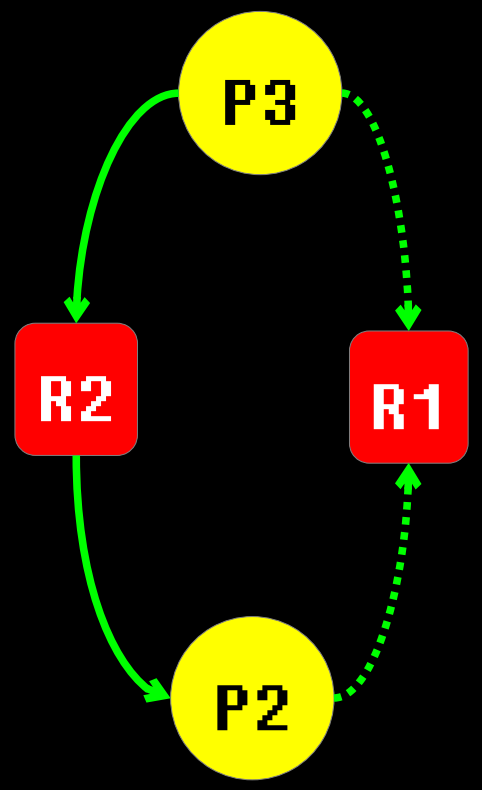
Evitar *deadlocks*

- Grafos de alocação
  - Direção processo-recurso: pedido e espera
  - Direção recurso-processo: posse



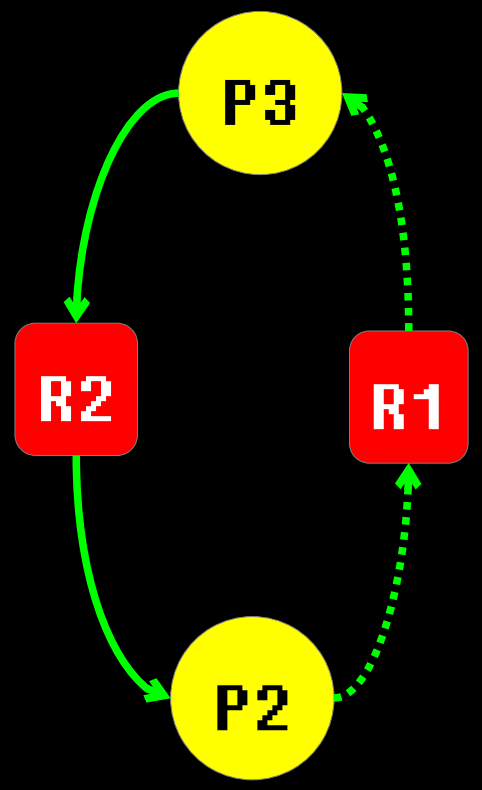
Evitar *deadlocks*

- Grafos de alocação
  - Direção processo-recurso: pedido e espera
  - Direção recurso-processo: posse



Evitar *deadlocks*

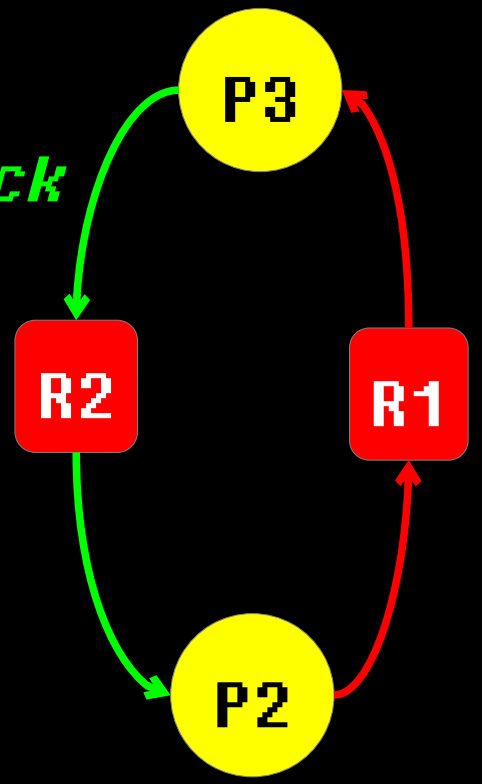
- Grafos de alocação
  - Direção processo-recurso: pedido e espera
  - Direção recurso-processo: posse



Evitar *deadlocks*

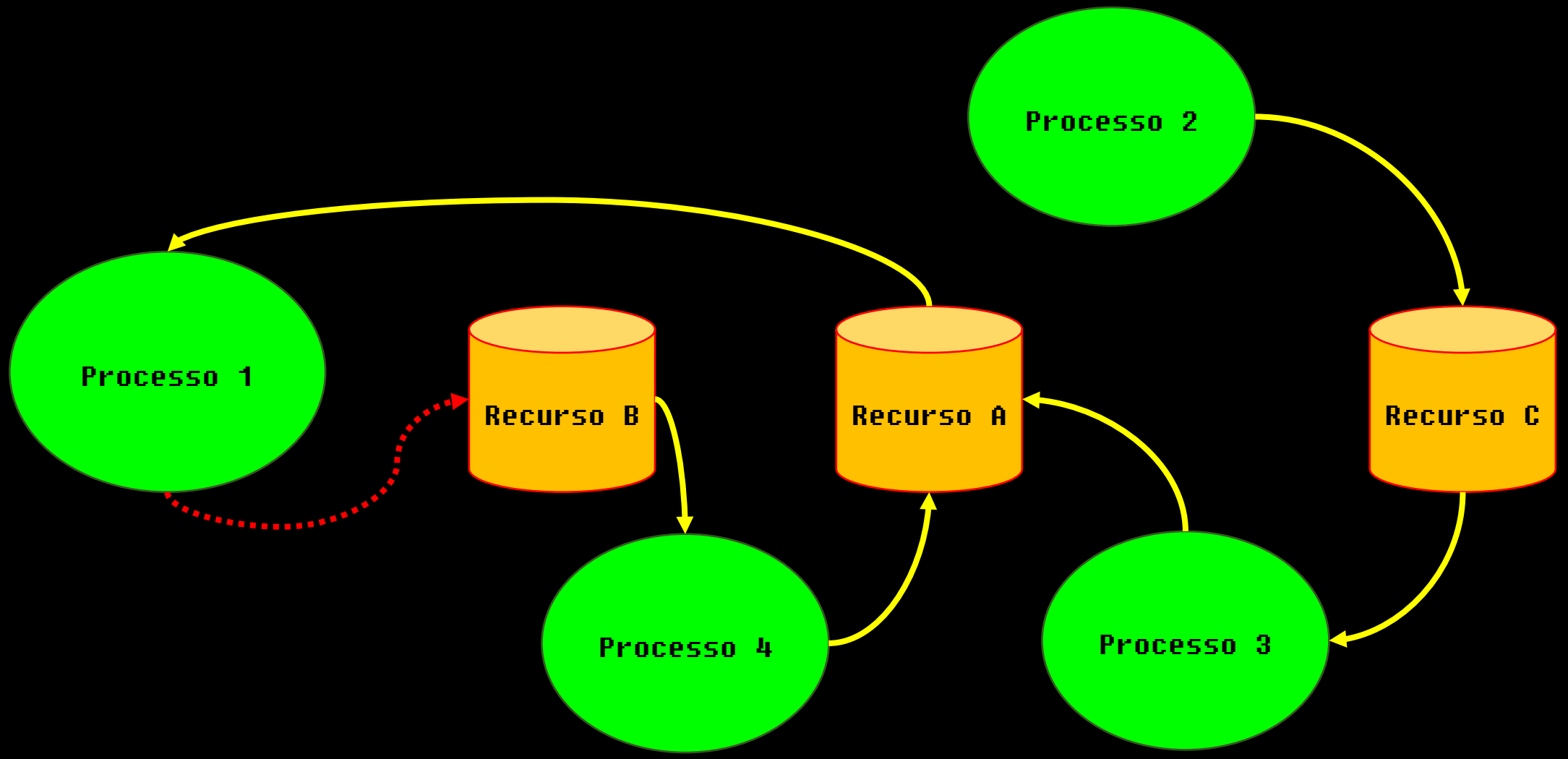
- Grafos de alocação
  - Direção processo-recurso: pedido e espera
  - Direção recurso-processo: posse

- Ciclo indica possível *deadlock*



Evitar *deadlocks*

> Grafos de alocação: na prática, mais complicado



Evitar *deadlocks*

- Estado seguro tem idéia simples: evitar que os recursos sejam fornecidos em quantidade superior ao necessário para terminar todos os processos.

- Alternativa 2: usar análise combinatória de cenários de cessão de recursos
  - Ex: algoritmo do banqueiro

Detecção e recuperação de deadlocks

Detecção e recuperação de *deadlocks*

- *Depois* que aconteceram, tentar solucionar o problema causado por um *deadlock*

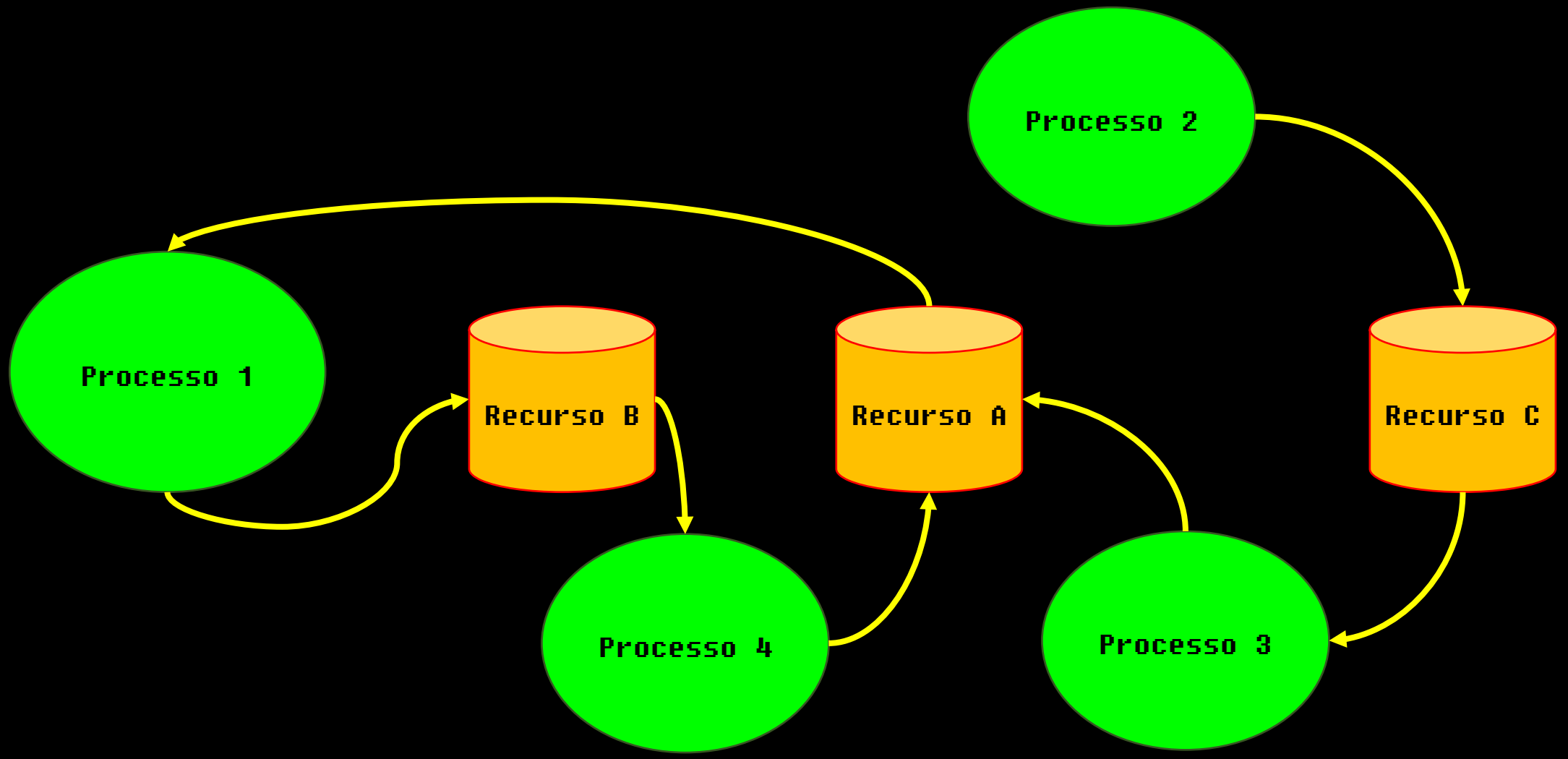
- *Destravar* o sistema e recuperar o estado consistente

Detecção e recuperação de *deadlocks*

- Alternativas para recuperação:
  - Suspensão e preempção
  - Retrocesso (*backtracking*)
  - Eliminação de processos

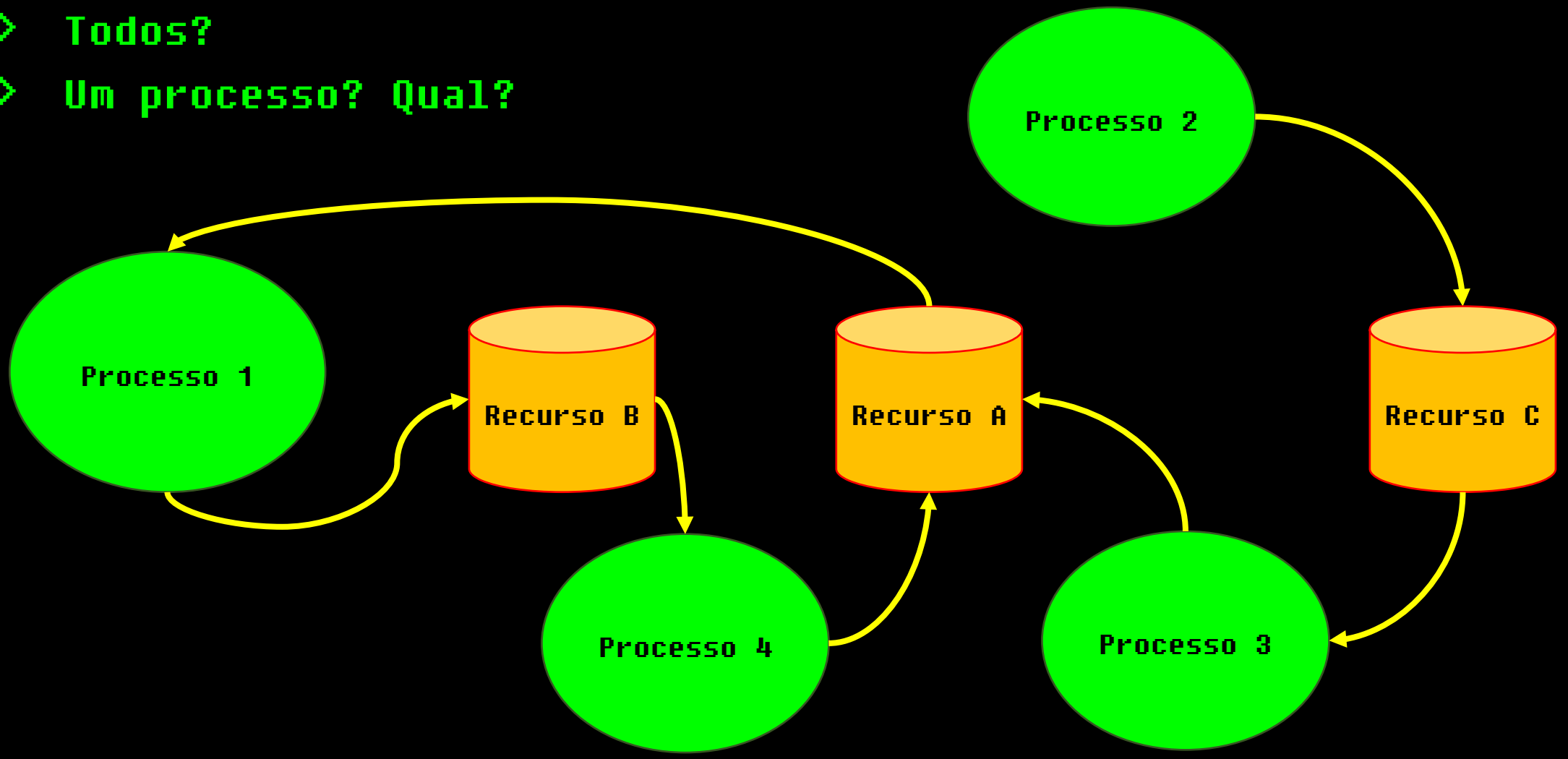
Detecção e recuperação de *deadlocks*

> Matar um processo para destravar o *deadlock*



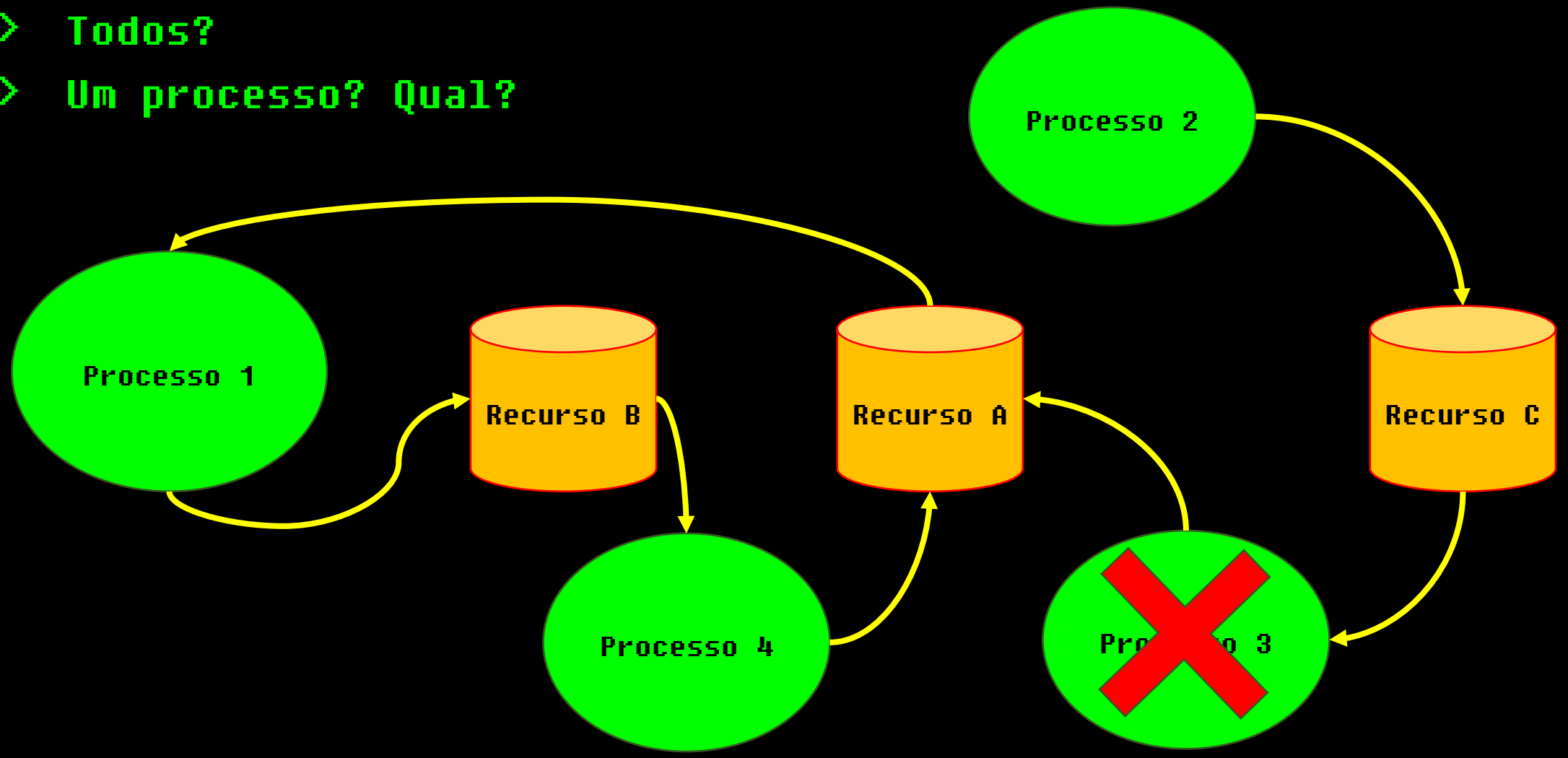
Detecção e recuperação de deadlocks

- Matar um processo para destravar o *deadlock*
  - Todos?
  - Um processo? Qual?



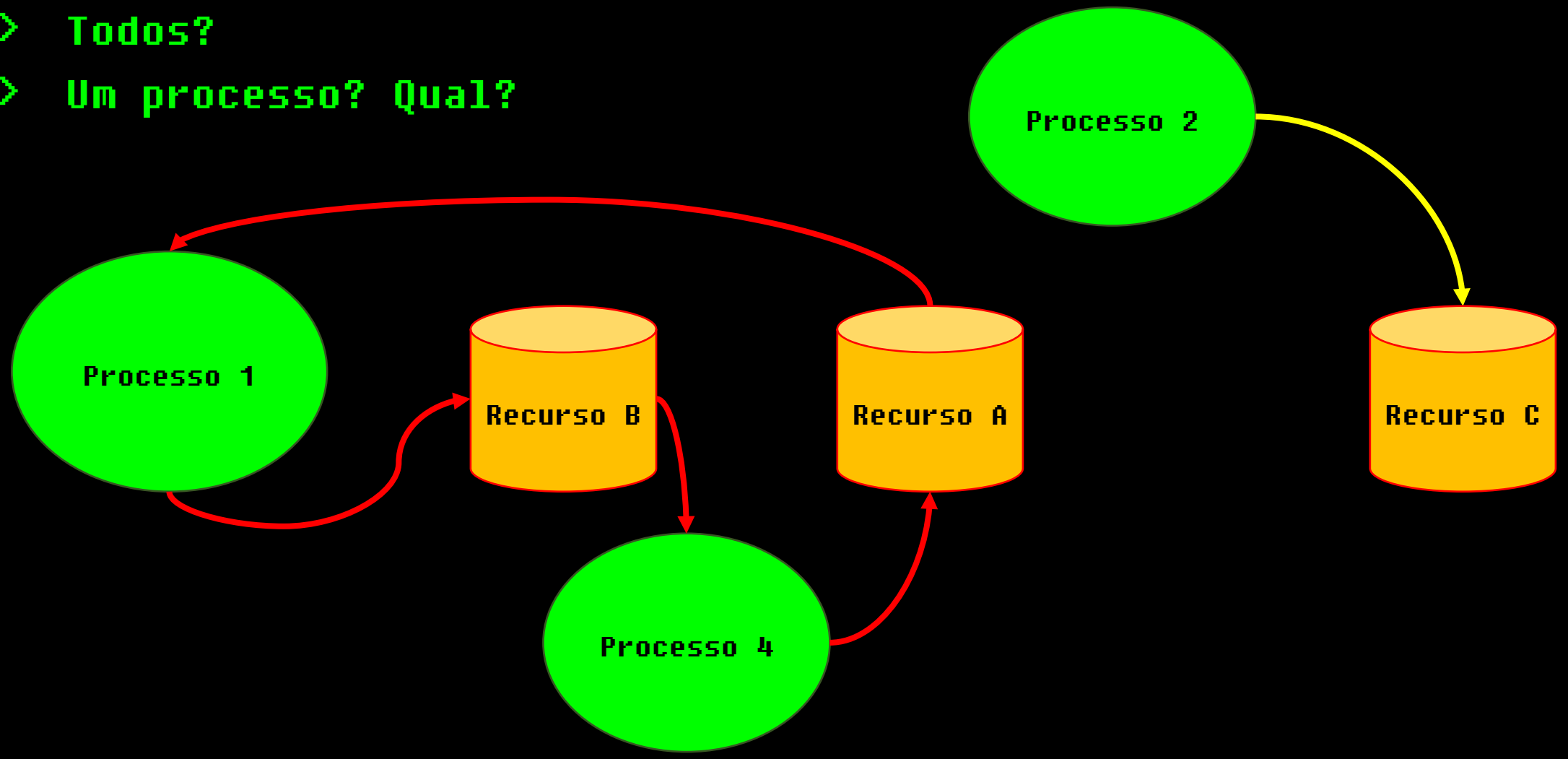
Detecção e recuperação de deadlocks

- Matar um processo para destravar o *deadlock*
  - Todos?
  - Um processo? Qual?



Detecção e recuperação de deadlocks

- Matar um processo para destravar o *deadlock*
  - Todos?
  - Um processo? Qual?



Detecção e recuperação de deadlocks

- Matar um processo para destravar o *deadlock*
  - Ao reiniciar o processo:
  - Refazer trabalho? Há inconsistências?

E agora?

Algoritmo do avestruz

Algoritmo do avestruz

> "Enfie a cabeça na areia e finja que não há nenhum problema".
> *(ok, sabemos que isso é uma lenda, mas a imagem ilustra bem a situação)*

SO e deadlocks

- Custo das alternativas para lidar com os deadlocks é considerado alto para o sistema.

- Situações comuns:

  - Programador faz programas sem *deadlocks*

  - Usuário é avisado de um potencial problema e resolve o que fazer se aparentemente houve um *deadlock*